

Project 2 rubric: Delivery

Baseline Requirements: 45 points

Autograder (35 Points)

Component	Points	Rubric
Get Artifact	3	Given a valid query, return all artifacts satisfying the indicated query.
Upload Artifact	3	Given a valid request with a valid URL string, the artifact is created successfully, and a valid response (201/202) is returned.
Ingest Artifact	3	Given a request with the URL field pointing to an ingestible artifact, the artifact is successfully uploaded.
Delete Artifact	2	Given a valid artifact ID, the delete endpoint removes the artifact.
Rate Artifact	4	Given a valid artifact ID, the endpoint returns the complete metric scores for the artifact.
Download Artifact	2	Given a valid artifact ID, returns the metadata and content for the artifact
Search for an artifact using regex	3	Given a regex string in query parameters, returns a list of artifacts that match that regex. It also avoids malicious regex by identifying and returning 400 for those (May not need 400 if using linear-time regex engines).
License Check	2	Given a GitHub URL, check if it has a valid license
Lineage	3	Given a model ID, check for the lineage graph of the model.
Cost	3	Given a valid artifact ID, it returns the total cost of an artifact and its dependencies
Reset to Default State	2	A call to the endpoint returns a successful response, and after the endpoint is called,

Component	Points	Rubric
		subsequent calls to get the list of artifacts should return an empty list
Concurrency	2	Concurrently, rate 14 models from 14 clients were completed successfully. No crash.
Frontend Auto-test	3	Valid frontend interactions and check if they are ADA-compliant using Lighthouse.
Sub-total	35	

Manual checks (5 points)

Test Evidence	3	3 points: Provided evidence that they performed manual or automated tests on all their reported features or provided justification while some features were untested. - Partial points if the evidence provided is not exhaustive.
LLM Usage	3	- LLMs are used to analyze the README or to analyze the relationship between artifacts. - Provided evidence that LLMs are used in the development phase to generate code or review PR - Provide evidence of intentional tuning of inference parameters (e.g, temperature, top_p, max_tokens). Includes structured prompts (e.g., defines role of LLM and user). Includes safeguards against bad outputs (e.g., hallucinations or formatting errors)
Browser-based Interface	4	- Provided evidence that they implemented a browser-based interface for their APIs. - Provides a usable web browser interface that exposes core system functionality for human users (upload/query/download), satisfying the accessibility/UI requirements. - Provided evidence that they implemented frontend automated tests, e.g., with Selenium - Provided evidence that they assessed their interface for ADA compliance.
Sub-total	10	Would be scaled down to 5 points

Track-specific Requirements: 10 points

Performance Track	10	• Appropriate API design and documentation as reflected in the OpenAPI specification changes • Measure throughput + mean, median, 99th percentile latency for 100 clients downloading Tiny-LLM from a registry with 500 models. Experiment design must be described. Must be triggerable from the system health dashboard. • Provide black-box measurements (client perspective) and white-box explanation (internal root causes). • Identify ≥ 2 performance bottlenecks, explain how they were found, how they were optimized, and show before/after measurements. • Modify the system so backend components (e.g., Lambda vs EC2, S3 vs DB) can be swapped through config. Measure their impact on latency & throughput.
Security Track	10	• Appropriate API design and documentation as reflected in the OpenAPI specification changes • Users authenticate using username + secure password. Authentication returns a token valid for 1000 API interactions or 10 hours. Multiple active tokens allowed per user. Passwords and authentication must be implemented using secure practices. Passwords must not be stored in plaintext. • The system must allow the uploading and updating of an arbitrary JavaScript program associated with a sensitive model. The model should only be downloaded if the program exists with a zero return code. • The system must store and return historical information about sensitive modules (what changed, when, and by whom). • The system must support an endpoint that returns a list of models that it suspects to be malicious.
High-assurance track	10	• $\geq 90\%$ line coverage from component tests alone; $\geq 95\%$ when including feature + end-to-end tests. • $\geq 80\%$ of all error messages must be triggered in the test suite. • Demonstrate robustness under failure of one AWS component (e.g., S3 outage). Test scenarios based on system models and provide improved error handling.

		• The test plan must discuss hermetic testing (SWEng@Google), where it is achieved, where not, and rationale. • Implement 3-stage atomic update groups: (1) Initiate empty transaction; (2) Append ingest/upload/update ops; (3) Execute atomically. Must prevent race conditions and inconsistent states.
Renegotiated	10	• Successfully renegotiated not completing additional requirements. • If done so, will be awarded (Autograder + manual score) / total * 15. That is the same percentage scored in other sections. • If renegotiated for partial tracks, then fallback to tracks with reduced requirements.

Manual Inspection for Engineering Practice: 10 points

Documentation	3	• 2 point: Has a README that explains the purpose of the project, including, e.g., how to configure it, deploy it, and interact with it. • 1 point: Three randomly chosen files all have introductory remarks at the top and some semblance of JavaDoc/PyDoc/etc. throughout
Effective communication	2	• Look at three randomly chosen files • 1 point: Are the names of variables and functions “pretty good” • 1 point: Is the style consistent across these files?
Appropriate CI/CD	3	• 1 point: Presence of YAML/etc. files encoding the CI/CD pipeline • 1 point: Successful implementation of CI (linting, testing, etc.) • 1 point: Successful implementation of CD (Automatic deployment of application to AWS. Full credit if much of this process is automated, and the rest is documented. Some manual steps, e.g. distributing tokens, are OK as long as they are documented)
Teamwork	2	• Each teammate contributed at least 100 lines of the code according to a simple “sum the lines/authors via git blame” check OR by hours reported in the report (with no follow-up complaints or side-channels from teammates)
Sub-total	10	